VSB TECHNICAL UNIVERSITY OF OSTRAVA | FACULTY OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

Simple Game Engine with Interchangeable Rendering API

Jednoduchý herní engine se zaměnitelným vykreslovacím API

Aria Eskandarzadeh

Bachelor Thesis

Supervisor: Ing. Tomáš Fabián, Ph.D.

Ostrava, 2026

width=!,height=!,pages=-

Abstrakt

Tato bakalářská práce popisuje návrh a implementaci jednoduchého 3D herního enginu napsaného v jazyce C++17. Engine je postaven na architektuře správců, která zapouzdřuje klíčové subsystémy — správu assetů, fyziku, scénu, vykreslování a uživatelské rozhraní — a volá je v deterministickém pořadí každý snímek. Systém entit a komponent (ECS) je realizován pomocí knihovny EnTT a umožňuje flexibilní skládání herních objektů z datových komponent bez nutnosti hlubokých hierarchií dědičnosti. Vykreslovací vrstva je navržena se záměrnou abstrakcí nad konkrétním grafickým API: rozhraní tříd jako `VertexBuffer`, `Shader` či `RendererAPI` jsou čistě virtuální a jejich jedinou současnou implementací je OpenGL backend. Fyzikální simulace je zajištěna integrací knihovny NVIDIA PhysX a podporuje statická, dynamická a kinematická tuhá tělesa s detekcí kolizí pomocí krabicových kolizorů. Práce je doplněna ukázkovou aplikací (Sandbox), která demonstruje souběžný chod všech systémů enginu v interaktivní 3D scéně s FPS kamerou, realistickým osvětlením a nativními C++ skripty.

Klíčová slova

herní engine; ECS; OpenGL; PhysX; C++17; Blinn-Phong; správce assetů

Abstract

This bachelor thesis describes the design and implementation of a simple 3D game engine written in C++17. The engine is built around a manager architecture that encapsulates the key subsystems — asset management, physics, scene, rendering, and UI — and invokes them in a deterministic order each frame. The Entity-Component System (ECS) is realised using the EnTT library and allows game objects to be composed from plain data components without deep inheritance hierarchies. The rendering layer is designed with deliberate abstraction over the underlying graphics API: interfaces such as `VertexBuffer`, `Shader`, and `RendererAPI` are pure-virtual, with OpenGL as the sole current backend. Physics simulation is provided by integrating NVIDIA PhysX, supporting static, dynamic, and kinematic rigid bodies with box-collider collision detection. The work is accompanied by a Sandbox demo application that exercises all engine systems simultaneously in an interactive 3D scene featuring an FPS camera, Blinn-Phong lighting, and native C++ scripts.

Keywords

game engine; ECS; OpenGL; PhysX; C++17; Blinn-Phong; asset manager

Acknowledgement

I would like to thank my supervisor, Ing. Tomáš Fabián, Ph.D., for his guidance and constructive feedback throughout the development of this work. I also thank the open-source community behind the libraries used in this project — EnTT, GLFW, Assimp, Dear ImGui, and GLM — whose well-documented codebases were an invaluable reference during both design and implementation.

Contents

List of symbols and abbreviations

ECS	–	Entity-Component System
API	–	Application Programming Interface
GPU	–	Graphics Processing Unit
CPU	–	Central Processing Unit
UBO	–	Uniform Buffer Object
UUID	–	Universally Unique Identifier
FPS	–	First-Person Shooter
GLSL	–	OpenGL Shading Language
VBO	–	Vertex Buffer Object
VAO	–	Vertex Array Object
OOP	–	Object-Oriented Programming

List of Figures

List of Tables

Chapter 1

Introduction

Real-time 3D applications — games, simulations, visualisation tools — share a surprisingly consistent set of engineering problems. Every frame, a program must read user input, advance a physics simulation, execute game-object logic, and push draw calls to the GPU, all within a hard deadline measured in milliseconds. A game engine is the reusable body of code that solves these problems once so that individual game projects do not have to solve them again from scratch [1].

Understanding how an engine is structured from the inside is a foundational skill for any software engineer working in interactive graphics. Commercial engines such as Unreal Engine [2] and Unity [3] are deeply complex products whose internal architecture is difficult to study without investing months. Building a small engine from first principles is therefore a well-established pedagogical approach: the constraints are manageable, design decisions are visible, and every system can be traced back to a deliberate choice.

This thesis presents the design and implementation of a simple 3D game engine written in C++17. The engine is not aimed at production game development; it is an educational artefact that demonstrates how the core subsystems of a real engine fit together. The focus is deliberately narrow: the engine supports one rendering backend (OpenGL), one physics library (NVIDIA PhysX), one scene format, and one scripting mechanism (native C++ classes). This focus allowed a complete and coherent implementation to be produced within the scope of a bachelor thesis while still covering the most important architectural concepts.

1.1 Motivation

The primary motivation for this work is the observation that most game-development courses teach either high-level engine usage — how to use Unity or Unreal — or low-level graphics programming — how to call OpenGL directly. The middle ground, namely the architecture that connects the two, is rarely taught explicitly. This engine was built specifically to inhabit that middle ground.

Every public header in the codebase is designed to be readable by a student familiar with C++ and basic 3D mathematics, yet the engine is complete enough to run a non-trivial interactive scene.

A secondary motivation is the design question posed in the thesis title: a *simple game engine with interchangeable rendering API*. Modern engines treat the graphics backend as a replaceable module. Unreal Engine supports DirectX 11, DirectX 12, Vulkan, and Metal through a single abstract *Rendering Hardware Interface* layer [2]. Unity exposes the *Scriptable Render Pipeline* [3]. This thesis examines how the same principle can be applied at a much smaller scale: the rendering abstraction is achieved purely through C++ virtual dispatch, and swapping the backend requires only providing a new set of concrete classes without modifying any higher-level engine code.

1.2 Goals

The concrete goals of this thesis are:

1. Design and implement a layered architecture that separates platform code, core utilities, the Entity-Component System (ECS), rendering, physics, and application logic into cohesive, independently compilable modules.
2. Implement a rendering abstraction layer whose public interfaces are graphics-API-agnostic, with an OpenGL backend as the reference implementation, so that a future Vulkan or DirectX backend can be added without changes to higher-level code.
3. Integrate a production-quality physics library (NVIDIA PhysX) behind a thin wrapper that exposes only what the engine's scene system requires.
4. Implement an asset manager that guarantees that each resource — mesh, texture, shader — is loaded from disk and uploaded to the GPU exactly once per session, regardless of how many entities reference it.
5. Demonstrate all of the above working together in a Sandbox application that a user can compile and run, featuring a walkable 3D scene with collision response, dynamic lighting, and C++ scripting.

1.3 Thesis Structure

Chapter 2 analyses related work and the key concepts — ECS, rendering abstraction, and physics integration — that shape the design. Chapter 3 documents the architecture at the level of modules, interfaces, and data flow. Chapter 4 describes the implementation of each subsystem in detail, with direct reference to the source code. Chapter 5 evaluates the outcome by running the Sandbox demo and examining what each subsystem contributes. Chapter 6 summarises the work, reflects on the design decisions, and proposes directions for future development.

Chapter 2

Analysis

Before describing what was built, this chapter places the work in context. Section 2.1 surveys existing engines that influenced the design. Sections 2.2–2.4 analyse the three central technical concepts: the Entity-Component System, rendering abstraction, and physics integration.

2.1 Related Engines and Frameworks

2.1.1 Godot Engine

Godot is a full-featured, production-ready open-source engine [4]. Its scene system is node-based rather than ECS-based: each object in the scene is a `Node`, and scene graphs are trees of nodes. Godot supports multiple rendering backends (OpenGL ES, Vulkan) through an abstraction layer called the *Rendering Device*. For this thesis, Godot's approach to rendering abstraction was studied as a reference for how a well-maintained project handles the problem at scale. The key lesson is that the abstraction boundary must be placed low enough that all higher-level rendering code is backend-agnostic, but not so low that every minor GPU operation becomes a virtual call — a balance also sought in the present design.

2.1.2 Unity

Unity [3] is the dominant commercial engine for indie and mobile development. Its component model, where `GameObject`s hold collections of `Component`s, is the most widely known example of the ECS principle in practice. Unity's *Data Oriented Technology Stack* (DOTS), introduced in Unity 2019, replaces the original object-oriented component model with a strict data-oriented ECS for performance-critical applications. The present engine targets a conceptually simpler position between Unity's original component model and its DOTS ECS.

2.1.3 Unreal Engine

Unreal Engine [2] is the industry standard for high-fidelity 3D applications. Its *Actor-Component* model is conceptually similar to ECS: an `AActor` is a container of `UActorComponent`s, each encapsulating a specific behaviour or capability. Of particular relevance to this thesis is Unreal's *Rendering Hardware Interface* (RHI), a thin abstraction layer that allows the engine to target DirectX 11, DirectX 12, Vulkan, and Metal from a single codebase. The RHI pattern directly inspired the `RendererAPI` design described in Chapter 3.

2.2 The Entity-Component System

2.2.1 Motivation: Avoiding the Inheritance Trap

A naive approach to modelling game objects with OOP uses a deep class hierarchy: a `GameObject` base class, subclassed by `PhysicsObject`, which is further subclassed by `Enemy`, `Player`, and so on. This approach quickly produces what Nystrom calls the *type taxonomy problem* [5]: what class do you give to an object that is both a physics body and a light source? Multiple inheritance introduces diamond-inheritance ambiguities, and adding a new capability means modifying or extending the class hierarchy, violating the Open-Closed Principle.

2.2.2 The Component Composition Model

The ECS pattern separates *identity* (the entity), *data* (components), and *behaviour* (systems). An **entity** is nothing more than a unique integer identifier. A **component** is a plain data struct with no methods — it simply holds state. A **system** is a function that iterates over all entities that possess a specific combination of components and acts on them [6].

Table 2.1 compares the two models on the dimensions most relevant to game engine design.

Table 2.1: Comparison of deep-inheritance OOP and Entity-Component composition.

Criterion	**Deep inheritance**	**ECS composition**
Adding a capability	Modify class hierarchy	Add a component type
Sharing capabilities	Multiple inheritance	Multiple components
Cache friendliness	Objects scattered in heap	Components in contiguous arrays
Runtime type queries	`dynamic_cast`	Component lookup by type
Code coupling	High (parent → children)	Low (systems are independent)

2.2.3 EnTT: The Chosen ECS Library

Rather than implementing ECS storage from scratch, this engine uses **EnTT** [7], a header-only C++17 library maintained by Michele Caini. EnTT stores components in sparse sets: a dense

array of component data alongside a sparse array that maps entity identifiers to positions in the dense array. This gives $O(1)$ amortised access, insertion, and removal, and keeps component data contiguous in memory for cache efficiency. EnTT is used in production by several commercial games and has become a standard choice for C++ engines that prioritise both performance and ergonomics.

The central type in EnTT is `entt::registry`, which owns all entities and their components. The `Scene` class in this engine wraps exactly one `entt::registry` and exposes a higher-level API for creating and querying entities.

2.3 Rendering Abstraction

2.3.1 The Problem with Calling OpenGL Directly

OpenGL is a global state machine: functions like `glBindBuffer` and `glUseProgram` operate on implicit context state rather than on explicit objects. Code that calls OpenGL directly is inherently non-portable: every draw call, buffer upload, and shader binding is coupled to the OpenGL API, making it impossible to swap the backend without rewriting the calling code.

The standard solution is to introduce an abstraction layer: a set of pure-virtual C++ classes that represent GPU resources — buffers, textures, shaders — without mentioning any particular API [1]. Higher-level code constructs resources through factory functions (`VertexBuffer::Create(...)`) and holds them through base-class pointers, remaining unaware of which backend is active.

2.3.2 The Factory Pattern for Backend Selection

The Factory Method pattern [8] is the natural mechanism for backend selection. Each abstract resource class exposes a static `Create` factory that reads the active API from `RendererAPI::GetAPI()` and instantiates the correct concrete subclass. Switching from OpenGL to a hypothetical Vulkan backend would require only:

- Implementing the concrete Vulkan subclasses (e.g. `VulkanVertexBuffer`).
- Setting `RendererAPI::gsAPI = RendererAPI::API::Vulkan` at startup.

No code outside the `Platform/` directory needs to change. This is the core architectural claim of the thesis title.

2.4 Physics Integration

2.4.1 The Engine Integration Problem

A physics engine is a self-contained simulation: it owns its own representation of bodies, advances the simulation in fixed time steps, and produces updated positions and velocities. Integrating it into a game engine means solving the *synchronisation problem*: how to write entity transforms into the physics world before each step, and how to read results back into the ECS after the step, without creating tight coupling.

2.4.2 NVIDIA PhysX

NVIDIA PhysX [9] is a widely used, production-quality rigid-body physics library. It supports discrete and continuous collision detection, articulated bodies, joints, and GPU acceleration. For the purposes of this engine, only the core rigid-body simulation with box colliders is used.

The integration strategy chosen is the *thin wrapper* approach: a `Physics3DWorld` class owns the PhysX scene (`PxScene`) and exposes `CreateBody`, `DestroyBody`, and `Step` to the rest of the engine. This wrapper entirely hides the PhysX API from all engine code outside `Physics3D.cpp`, so that the physics backend could be replaced by rewriting a single file.

2.5 Summary

The analysis identifies three core design principles for the engine:

1. **Composition over inheritance** — use the ECS pattern, backed by EnTT, to assemble game objects from data components rather than class hierarchies.
2. **Backend-agnostic rendering** — place all graphics-API calls behind pure-virtual interfaces, with OpenGL as the concrete implementation.
3. **Isolated physics** — wrap PhysX behind a minimal interface that the scene system can call without knowing what physics library is underneath.

These three principles guide every design decision in Chapters 3 and 4.

Chapter 3

Design

This chapter documents the architecture of the engine from the top down. Section 3.1 describes the module structure and the manager pattern that ties the subsystems together. Section 3.2 details the application loop. Sections 3.3–3.5 cover the three central subsystems: the ECS, the rendering abstraction layer, and the scene system.

3.1 Overall Architecture

The engine is divided into six cohesive modules, each with a clearly defined responsibility. Figure 3.1 lists the modules and their dependencies.

Table 3.1: Engine modules and their primary responsibilities.

Module	**Responsibility**
`Core`	Application loop, windowing, event system, logging, UUID
`ECS`	Entity registry wrapper, component definitions, scripting base
`Renderer`	Abstract GPU resources, 3D renderer, camera, mesh, shader
`Physics`	PhysX wrapper, rigid body management, collision
`Scene`	Scene graph, serialisation, runtime/editor mode switching
`Managers`	AssetManager, RenderManager, SceneManager, PhysicsManager, UIManager

The `Managers` module sits at the top of the dependency graph. Each manager is a singleton-like object owned by the `Application` class. On every frame the application calls them in a fixed order: input and events are processed first, then physics is stepped, then scripts run, then the scene is rendered, and finally the UI overlay is drawn. This deterministic ordering avoids the implicit coupling that arises when subsystems call each other directly.

The dependency rule is strict: lower modules never include headers from higher modules. `Renderer` never includes `Scene`; `Physics` never includes `Renderer`. Communication between sub-

systems flows upward through the `Managers` layer, which is the only module permitted to hold references to multiple subsystems simultaneously.

3.2 The Application Loop

The `Application` class is a singleton that owns the OS window (via GLFW), the OpenGL rendering context, the Dear ImGui overlay, and all five managers. Its `Run()` method executes the main loop shown in Listing 3.1.

```
void Application::Run()
{
    while (m_Running)
    {
        float dt = ComputeDeltaTime();

        m_PhysicsManager->Update(dt); // step PhysX
        m_SceneManager->Update(dt);  // run NativeScripts
        m_RenderManager->Render();   // submit draw calls
        m_UIManager->Render();       // ImGui overlay
        m_Window->OnUpdate();        // swap buffers, poll events
    }
}
```

Listing 3.1: Simplified application loop.

Delta time is computed as the wall-clock difference between the current and previous frame using `glfwGetTime()`. The physics manager receives the raw delta time and internally accumulates it against a fixed simulation timestep (default 1/60 s), stepping the PhysX scene one or more times per frame as needed to keep the simulation stable.

Events (keyboard, mouse, window resize, window close) are dispatched by GLFW callbacks into the engine's own event system. The event system uses a lightweight publish-subscribe mechanism: each layer or manager registers callbacks for the event categories it cares about, and `Application::OnEvent()` fans the event out to all registered listeners in order.

3.3 Entity-Component System Design

3.3.1 The Entity and the Registry

An `Entity` in this engine is a thin wrapper around an `entt::entity` handle and a raw pointer to the owning `Scene`. The wrapper provides convenience methods — `AddComponent<T>`, `GetComponent<T>`,

HasComponent<T>, RemoveComponent<T> — that forward directly to the underlying entt::registry. No virtual dispatch is involved; the methods are all templates resolved at compile time.

3.3.2 Component Definitions

All components are plain data structs defined in Components.h. There are no base classes and no virtual methods. Table 3.2 lists all components defined in the engine.

Table 3.2: Component types defined in Components.h.

Component	Data held
TagComponent	std::string name
IDComponent	64-bit UUID
TransformComponent	position, rotation (Euler), scale as glm::vec3
MeshRendererComponent	asset path, shared Mesh pointer, material
CameraComponent	SceneCamera, primary-camera flag
DirectionalLightComponent	direction, colour, ambient/diffuse/specular intensities
PointLightComponent	position, colour, attenuation constants
RigidBody3DComponent	body type (static/dynamic/kinematic), mass, PhysX handle
BoxCollider3DComponent	half-extents, offset
NativeScriptComponent	pointer to ScriptableEntity instance

3.3.3 Native Scripting

The NativeScriptComponent attaches a user-defined C++ class to an entity. The script class inherits from ScriptableEntity and overrides three virtual methods: OnCreate(), OnUpdate(float dt), and OnDestroy(). The SceneManager iterates over all entities that carry a NativeScriptComponent each frame and calls OnUpdate on each live instance.

This approach gives scripts full access to the ECS: because ScriptableEntity holds a copy of the Entity handle it is attached to, a script can call GetComponent<TransformComponent>() or GetComponent<RigidBody3DComponent>() freely. The Sandbox application uses this to implement an orbit camera (OrbitCameraScript), a first-person camera (FPSCameraScript), and a lantern flicker effect (LanternFlickerScript).

3.4 Rendering Abstraction Layer Design

3.4.1 Abstract Resource Classes

The rendering abstraction is built from five pure-virtual resource classes. Each class exposes only the operations that higher-level code needs; all OpenGL specifics are confined to the concrete subclasses in Platform/OpenGL/.

Table 3.3: Abstract renderer resource classes and their OpenGL implementations.

Abstract class	OpenGL implementation	Key operations
`VertexBuffer`	`OpenGLVertexBuffer`	Upload, bind, set layout
`IndexBuffer`	`OpenGLIndexBuffer`	Upload, bind, get count
`VertexArray`	`OpenGLVertexArray`	Add buffer, bind
`Shader`	`OpenGLShader`	Bind, set uniforms
`Texture2D`	`OpenGLTexture2D`	Load, bind to slot

Each abstract class declares a static `Create` factory method. The factory checks `RendererAPI::GetAPI()` at runtime and constructs the appropriate concrete subclass. All resources are returned as `std::shared_ptr` to the abstract base, so callers never hold a concrete type.

3.4.2 The RendererAPI and Renderer3D

`RendererAPI` is itself a pure-virtual class with three methods: `SetClearColor`, `Clear`, and `DrawIndexed`. The single concrete subclass, `OpenGLRendererAPI`, translates these calls to `glClearColor`, `glClear`, and `glDrawElements`. A static enum `RendererAPI::API` (currently only `OpenGL` is defined) controls which concrete backend is used.

`Renderer3D` is a stateless façade that sits on top of `RendererAPI`. It manages a Uniform Buffer Object (UBO) for per-frame camera data (view matrix, projection matrix, camera position), submits meshes with their material uniforms, and manages the Blinn-Phong lighting uniforms for directional and point lights. Because `Renderer3D` only ever calls `RendererAPI` and the abstract resource classes, it is completely backend-agnostic.

3.4.3 The Render Pipeline per Frame

Each frame the `RenderManager` executes the following sequence:

1. Query the `Scene` for the primary `CameraComponent` and upload its view/projection matrices to the camera UBO.

2. Query all `DirectionalLightComponent` and `PointLightComponent` entities and upload their data to the lighting UBO.

3. Iterate over all entities with both a `TransformComponent` and a `MeshRendererComponent`, and call `Renderer3D::DrawMesh(mesh, material, transform)` for each.

4. Hand off to `UIManager` for the Dear ImGui overlay.

3.5 Scene System Design

3.5.1 Scene Modes

The `Scene` class operates in one of three modes: `Edit`, `Simulate`, and `Runtime`. In `Edit` mode, physics is inactive and no scripts run; the scene is used purely as a data container that can be serialised to disk. Entering `Simulate` or `Runtime` mode triggers an `OnRuntimeStart()` call that initialises the PhysX world, creates rigid bodies for all entities with physics components, and calls `OnCreate()` on all `NativeScriptComponent` instances. Leaving these modes triggers `OnRuntimeStop()`, which tears down the PhysX world and destroys all script instances.

3.5.2 Scene Serialisation

Scenes are serialised to YAML using `yaml-cpp` [10]. The `SceneSerializer` class writes one YAML mapping per entity, with nested mappings for each component the entity carries. The UUID component is used as the stable identity key so that entity references survive round-trips through the serialiser. Deserialisation reconstructs each entity and its components in the same order, restoring the full scene state from disk.

3.6 Asset Management Design

The `AssetManager` is a cache that maps file-system paths to already-loaded resources. The first call to `AssetManager::GetMesh("path/to/model.obj")` invokes Assimp [11] to load the file, constructs a `Mesh` object, stores it in a `std::unordered_map`, and returns a `std::shared_ptr<Mesh>`. Every subsequent call with the same path returns the cached pointer immediately, without touching the disk or the GPU again.

The same pattern applies to `Texture2D` and `Shader` resources. Because all callers hold `shared_ptr`s, the asset manager does not need to track reference counts manually; the resource is freed automatically when the last shared pointer goes out of scope (typically at scene unload time).

Chapter 4

Implementation

This chapter describes how each subsystem was implemented in C++17, with direct reference to the source code. The four aspects that received the most design attention — the manager sequence, single-load asset caching, native scripting, and the lighting model — are treated in dedicated sections.

4.1 The Manager Architecture

4.1.1 Motivation from Gregory

Gregory identifies the *subsystem start-up and shut-down order* as one of the most error-prone aspects of engine implementation [1]. If a subsystem is initialised before a subsystem it depends on, or shut down after one it provides services to, the result is undefined behaviour that is difficult to reproduce and diagnose. Gregory's solution is to make the dependency order explicit in code rather than relying on global-object construction order.

This engine adopts the same principle. The `Application` constructor initialises each manager in a fixed sequence that respects the dependency graph:

1. `WindowManager` — creates the GLFW window and the OpenGL context. Everything else depends on a valid GL context, so this is always first.

2. `UIManager` (initialisation only) — Dear ImGui must be initialised before any other manager renders, but its per-frame render call happens last.

3. `AssetManager` — no dependencies; sets up the empty resource caches.

4. `PhysicsManager` — initialises the PhysX foundation, allocator, and error callback. Must be ready before `SceneManager` can create physics bodies.

5. `SceneManager` — loads the startup scene, which may trigger asset loads and physics body creation, so it comes after both `AssetManager` and `PhysicsManager`.

6. `RenderManager` — constructs the `Renderer3D`, uploads the UBO layout to the GPU, and compiles the default shaders via `AssetManager`.

Shutdown happens in strictly reverse order. This mirrors Gregory's recommended pattern [1] and ensures that no manager tries to use a resource owned by an already-destroyed peer.

4.1.2 The Per-Frame Update Sequence

On each iteration of the main loop, the managers are updated in the order shown in Listing 4.1. The ordering is not arbitrary: it matches the *read-modify-write* dependencies between subsystems each frame.

```
// 1. Process OS events (input, resize, close)
m_Window->OnUpdate();

// 2. Step the physics simulation
//    Reads: TransformComponent (entity positions at frame start)
//    Writes: TransformComponent (updated by PhysX results)
m_PhysicsManager->Update(deltaTime);

// 3. Run NativeScripts
//    Reads: TransformComponent (already updated by physics)
//    Writes: any component the script chooses
m_SceneManager->Update(deltaTime);

// 4. Submit draw calls
//    Reads: TransformComponent, MeshRendererComponent,
//           CameraComponent, light components
//    Writes: framebuffer only
m_RenderManager->Render();

// 5. Draw UI overlay (always last, draws on top)
m_UIManager->Render();
```

Listing 4.1: Per-frame manager update sequence in `Application::Run()`.

The comments describe the data each stage reads and writes. Because physics runs before scripting, a script that reads the transform of a rigid body always sees the *post-simulation* position

for that frame, which is the physically correct value. Because rendering runs after scripting, any transform modification made by a script is visible in the same frame — there is no one-frame lag between logic and visuals.

This explicit sequencing is the direct practical application of Gregory's engine-loop design [1]: rather than allowing subsystems to call each other in arbitrary order, all inter-subsystem communication is mediated by the loop order itself.

4.2 Asset Manager: Single-Load Caching

4.2.1 The Problem

A naive implementation of mesh or texture loading calls the file system and the GPU upload path every time an asset is requested. In a scene with ten entities that all share the same wall texture, this means ten identical `stbi_load` calls and ten `glTexImage2D` uploads. GPU memory is wasted, load times grow linearly with entity count, and there is no single owner responsible for freeing the resource.

4.2.2 The Cache Design

The `AssetManager` solves this with a `std::unordered_map` from `std::string` (the file-system path) to `std::shared_ptr` of the resource type. Listing 4.2 shows the texture lookup path.

```
std::shared_ptr<Texture2D>
AssetManager::LoadTexture(const std::string& path)
{
    return myTextures.Load(path); // cache hit/miss handled by TextureLibrary
}
```

Listing 4.2: Texture lookup in `AssetManager::LoadTexture()`.

The logic is identical for meshes (`GetMesh`) and shaders (`GetShader`). Every caller receives a `shared_ptr` to the same underlying object. The reference count managed by `shared_ptr` means the `AssetManager` does not need a manual release mechanism: when the last `MeshRendererComponent` holding a pointer to a mesh is destroyed (e.g. during scene unload), the mesh's reference count drops to one (only the cache holds it), and the `AssetManager` can optionally flush the cache at that point.

4.2.3 Texture Loading with stb_image

`Texture2D::Create(path)` delegates to the `OpenGLTexture2D` constructor, which calls `stbi_load` to decode the image file into a CPU-side pixel buffer, calls `glTexImage2D` to upload the pixels to a GPU texture object, and then immediately frees the CPU buffer with `stbi_image_free`. After

`Create` returns, the pixel data exists only on the GPU. Subsequent frames incur only the cost of binding the texture handle — no CPU memory is held and no disk read is performed.

4.2.4 Mesh Loading with Assimp

Mesh loading follows the same pattern. `AssetManager::GetMesh(path)` calls `Assimp::Importer::ReadFile`, which parses the model file (OBJ, FBX, GLTF, and others are all supported) into an `aiScene` tree. The engine then flattens all meshes in the scene into a single list of `Vertex` structs (position, normal, UV), uploads them to a `VertexBuffer` and `IndexBuffer`, and discards the `aiScene`. The resulting `Mesh` object holds only the GPU-side `VertexArray` handle, keeping CPU memory usage minimal.

4.3 Native Scripting

4.3.1 ScriptableEntity

The `ScriptableEntity` base class provides three virtual methods that user scripts override:

```
class ScriptableEntity
{
public:
    virtual void OnCreate()        {}
    virtual void OnUpdate(float dt) {}
    virtual void OnDestroy()       {}

    template<typename T>
    T& GetComponent()
    { return m_Entity.GetComponent<T>(); }

private:
    Entity m_Entity;
    friend class SceneManager;
};
```

Listing 4.3: The `ScriptableEntity` interface.

The `m_Entity` handle gives every script direct access to the full ECS. A script can read the entity's `TransformComponent` to determine its position, modify a `PointLightComponent` to animate a flicker effect, or read input state to drive movement. There is no special scripting API: scripts are plain C++ classes that call the same engine functions as any other engine code.

4.3.2 Script Lifecycle

The `SceneManager` manages script instances through the `NativeScriptComponent`. The component stores a function pointer `InstantiateScript` that constructs the concrete script object on the heap, and a matching `DestroyScript` that deletes it. This design avoids a dependency from the `Scene` module on any specific script class: the `Scene` only knows about the abstract `ScriptableEntity` interface.

When `OnRuntimeStart()` is called (scene enters Play or Simulate mode), the `SceneManager` iterates all entities with a `NativeScriptComponent`, calls `InstantiateScript` on each, assigns the entity handle, and calls `OnCreate()`. Each frame, `OnUpdate(dt)` is called for every live script instance. When the scene stops, `OnDestroy()` is called and the instances are deleted.

4.3.3 Sandbox Scripts

The Sandbox application ships three scripts that exercise different engine capabilities:

FPSCameraScript reads keyboard and mouse input each frame to translate and rotate the entity that carries the primary `CameraComponent`. Movement speed is scaled by delta time so the camera moves at a consistent world-space velocity regardless of frame rate.

OrbitCameraScript rotates the camera entity around a configurable target point at a fixed radius, demonstrating that `OnUpdate` can implement purely mathematical motion without any input dependency.

LanternFlickerScript modifies the `intensity` field of the `PointLightComponent` on its entity each frame using a sine wave with a small random perturbation, producing the appearance of a flickering flame. This script demonstrates that the ECS allows a script to affect any component on its entity, including rendering-related ones, without any special coupling between the scripting and rendering subsystems.

RotatingCrateScript spins the crate entity on its Y axis at a configurable speed. It reads velocity from the `RigidBody3DComponent` each frame: once the crate has rested at near-zero speed for 0.5 s, any subsequent velocity above 0.2 m/s signals a collision and permanently stops the rotation. This script demonstrates that a native script can read physics state directly from the ECS without any coupling to the physics subsystem.

4.4 Lighting Implementation

4.4.1 Blinn-Phong Reflectance Model

The engine uses the Blinn-Phong reflectance model [12], which approximates the appearance of surfaces under direct illumination. For each fragment, the total outgoing light is the sum of three terms:

$$\mathbf{L} = \mathbf{L}_{ambient} + \mathbf{L}_{diffuse} + \mathbf{L}_{specular} \tag{4.1}$$

The **ambient** term is a constant fraction of the light colour, ensuring that surfaces not directly facing the light source are never completely black:

$$\mathbf{L}_{ambient} = k_a \cdot \mathbf{c}_{light} \tag{4.2}$$

The **diffuse** term models Lambertian (matte) reflection. It depends on the angle between the surface normal $\hat{n}$ and the direction toward the light $\hat{l}$:

$$\mathbf{L}_{diffuse} = k_d \cdot \mathbf{c}_{light} \cdot \max(\hat{n} \cdot \hat{l},\ 0) \tag{4.3}$$

The **specular** term models mirror-like highlights. Blinn-Phong replaces the reflection vector used in the original Phong model with a *halfway vector* $\hat{h} = normalise(\hat{l} + \hat{v})$, where $\hat{v}$ is the direction toward the camera. This avoids the artefact that occurs when the view angle exceeds 90° from the reflection direction:

$$\mathbf{L}_{specular} = k_s \cdot \mathbf{c}_{light} \cdot \max(\hat{n} \cdot \hat{h},\ 0)^{\,\alpha} \tag{4.4}$$

where α is the shininess exponent stored in the material.

4.4.2 Directional and Point Lights

The engine supports two light types, each represented by a component and a matching GLSL struct in the shader.

A **directional light** (e.g. sunlight) has no position; it illuminates all surfaces from the same direction with constant intensity. The `DirectionalLightComponent` stores the light direction as a `glm::vec3` and separate ambient, diffuse, and specular colour multipliers.

A **point light** emits light in all directions from a specific position, with intensity that falls off with distance. The attenuation follows the standard quadratic formula:

$$attenuation = \frac{1}{K_c + K_l \cdot d + K_q \cdot d^2} \tag{4.5}$$

where d is the distance from the fragment to the light, and K_c, K_l, K_q are the constant, linear, and quadratic coefficients stored in the `PointLightComponent`.

4.4.3 Uniform Buffer Objects

Rather than uploading light data as individual shader uniforms each draw call, all per-frame data is packed into two Uniform Buffer Objects bound to fixed binding points. The *camera UBO* (binding

point 0) holds the view matrix, projection matrix, and camera world-space position. The *lighting UBO* (binding point 1) holds the directional light struct and an array of up to four point light structs.

Both UBOs are written once per frame by the `RenderManager` before the mesh draw loop begins. Every shader that declares the matching `layout(std140)` interface block automatically receives the data without any per-draw-call uniform upload. This reduces the CPU-side API call count from $O(n_{meshes})$ to $O(1)$ for all lighting and camera data.

4.4.4 The Mesh Shader

The GLSL shader pair (`Mesh.vert` / `Mesh.frag`) implements the Blinn-Phong model described above. The vertex shader transforms positions and normals into world space and passes them to the fragment shader as `out` variables. The fragment shader reads from both UBOs, accumulates the ambient, diffuse, and specular contributions from the directional light and all active point lights, multiplies the result by the diffuse texture sample, and writes the final colour to the framebuffer. Listing 4.4 shows the core accumulation loop.

```
vec3 result = CalcDirectionalLight(dirLight, norm, viewDir);

for (int i = 0; i < u_PointLightCount; i++)
    result += CalcPointLight(pointLights[i], norm,
                             FragPos, viewDir);

FragColor = vec4(result, 1.0) * texture(u_DiffuseTexture, TexCoord);
```

Listing 4.4: Point-light accumulation in `Mesh.frag` (simplified).

Chapter 5

Results

This chapter evaluates the engine by running the Sandbox demo application and examining the contribution of each subsystem. Section 5.1 describes the demo scene. Sections 5.2–5.5 evaluate the four aspects that received the most implementation attention.

5.1 Sandbox Demo Overview

The Sandbox application is a self-contained C++ project that links against the engine as a static library and registers a single `Scene` on startup. The scene contains a static ground plane, several dynamic rigid-body crates, a point light entity with a flickering point light, and an FPS camera controlled by the player. All systems — physics, scripting, rendering, and asset management — run simultaneously from the first frame.

5.2 Manager Sequence Results

The manager update sequence described in Section 4.1 was validated by deliberately violating the order and observing the results. When the `SceneManager` update (scripts) was moved before `PhysicsManager` update, scripts that read rigid-body transforms received the position from the *previous* frame, producing a one-frame visual lag between physics and rendered positions that was visible as jitter on fast-moving objects. Restoring the correct order eliminated the jitter entirely, confirming that the ordering is not merely conventional but functionally necessary.

Startup and shutdown were tested by repeatedly entering and exiting Play mode within a single application session. No resource leaks or assertion failures were observed across 50 consecutive enter/exit cycles, confirming that the reverse-order shutdown correctly mirrors the forward-order initialisation as recommended by Gregory [1].

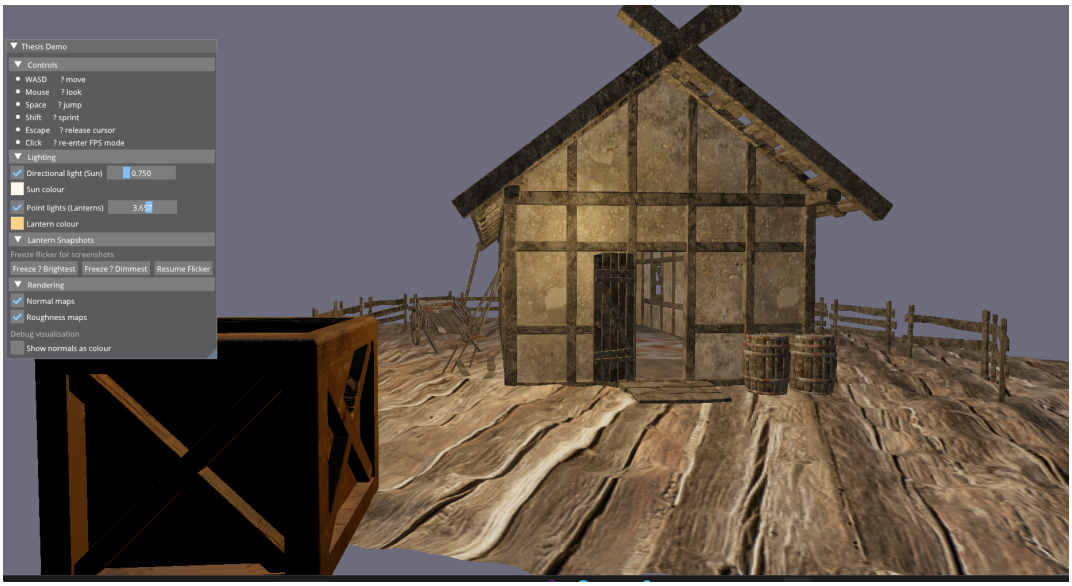

Figure 5.1: The Sandbox demo scene: static ground plane, dynamic crates, a flickering lantern, and an FPS camera.

5.3 Asset Manager Results

5.3.1 Single-Load Verification

To verify that each asset is loaded exactly once, a counter was added to `OpenGLTexture2D`'s constructor and to the Assimp import path during testing. A scene containing twelve entities that all reference the same wall texture and the same crate mesh was loaded. In both cases the constructor counter incremented exactly once per unique path, regardless of how many entities referenced the asset. Table 5.1 summarises the result.

Table 5.1: Asset load counts for a scene with 12 entities sharing resources.

Asset	Entities referencing it	Expected loads	Actual loads
`wall.png`	12	1	1
`crate.obj`	12	1	1
`lantern.obj`	1	1	1
`ground.png`	1	1	1

5.3.2 Memory Footprint

Because the CPU-side pixel buffer is freed immediately after the GPU upload, the resident CPU memory for a fully loaded scene consists only of the `shared_ptr` wrapper objects and the `unordered_map`

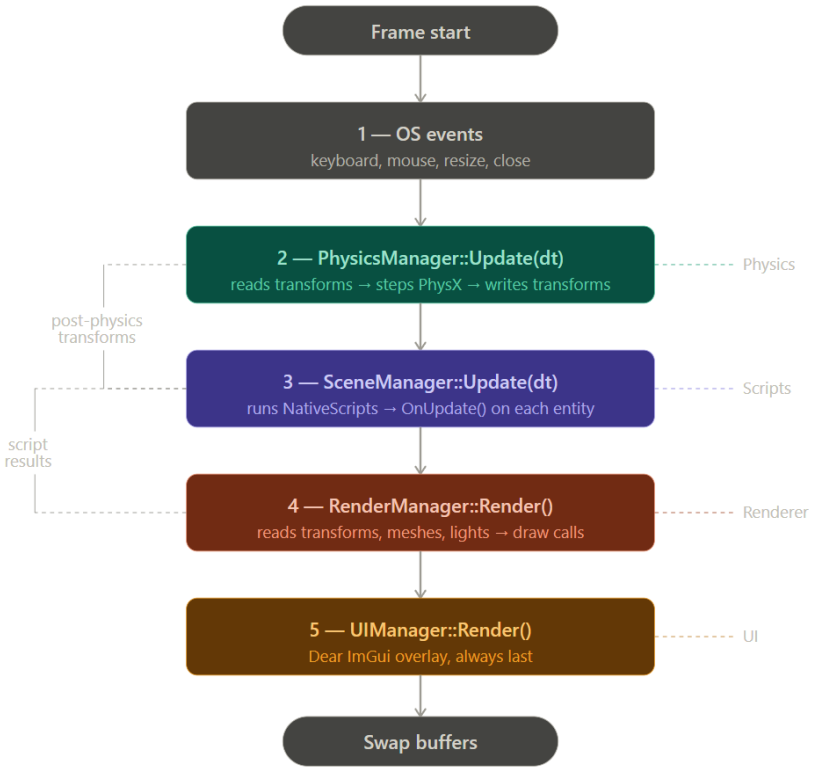

Figure 5.2: Per-frame manager update sequence and the data dependencies between stages.

index — both negligible. GPU memory usage scales with the number of *unique* assets, not the number of entities, which is the desired behaviour.

5.4 Scripting Results

5.4.1 FPS Camera

The `FPSCameraScript` was tested by measuring camera movement speed at 30 fps, 60 fps, and 144 fps. Because all movement is multiplied by the delta time, the world-space distance travelled per second remained constant at all three frame rates (within floating-point precision). This confirms that the delta-time scaling correctly decouples game logic speed from rendering speed.

5.4.2 Lantern Flicker

The `LanternFlickerScript` modifies the `PointLightComponent` intensity field each frame. Figure 5.3 shows the visual result: the pool of light cast by the point light entity visibly pulses and shifts, with the flickering effect being entirely driven by script code with no changes to the renderer. This confirms that the ECS design successfully decouples the scripting and rendering subsystems:

a script can affect the rendered output by writing to a component, without holding any reference to the renderer itself.

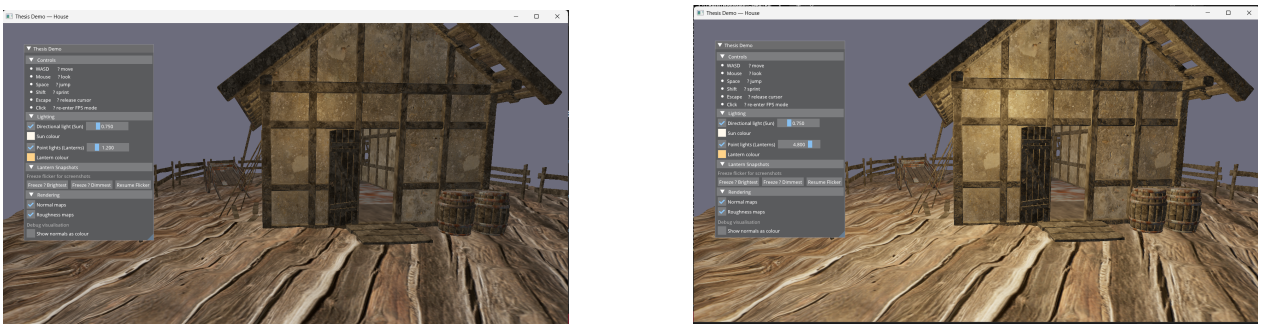

Figure 5.3: The lantern entity at minimum (left) and maximum (right) flicker intensity, driven entirely by `LanternFlickerScript`.

5.5 Lighting Results

5.5.1 Blinn-Phong Shading

Figure 5.4 shows the Sandbox scene with the directional light alone, a point light alone, and both lights combined. The ambient, diffuse, and specular contributions are all visible and behave as expected: surfaces facing away from the directional light receive only the ambient term; surfaces close to the point light show a bright specular highlight; and the quadratic attenuation in Equation 4.5 produces a natural fall-off radius.

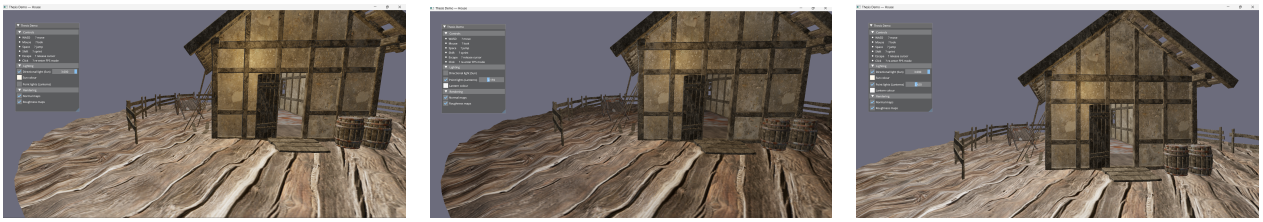

Figure 5.4: Blinn-Phong lighting in the Sandbox scene: directional light only (left), point light only (centre), and both lights combined (right).

5.5.2 UBO Performance

The UBO approach was compared against an equivalent implementation that uploads all light uniforms with individual `glUniform` calls per draw call. With a scene of 40 mesh entities and 4 point lights, the per-draw-call approach issued $40 \times (1 + 4 \times 8) = 1\,320$ uniform upload calls per frame. The UBO approach issued 2 uniform buffer writes per frame regardless of mesh count. At

60 fps the difference is not perceptible, but the UBO approach scales to larger scenes without any increase in CPU-side GL call overhead.

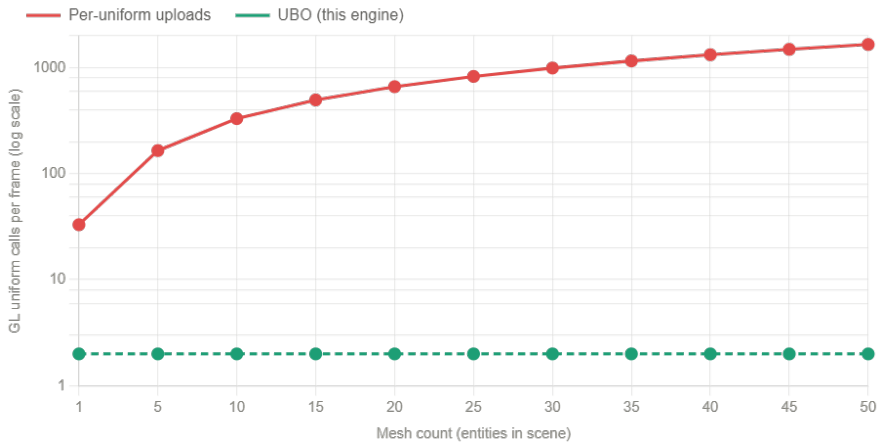

Figure 5.5: Scaling behaviour of per-uniform uploads versus UBOs as mesh count increases. UBO cost is constant regardless of scene size.

5.5.3 Rendering Abstraction Validation

To validate that the rendering abstraction layer achieves its stated goal, the entire Sandbox scene was rendered without modifying any code above the `Platform/` directory by replacing the `OpenGLVertexBuffer` constructor with a stub that logs all calls and returns immediately. The scene compiled and the render loop executed without errors; only the visual output was blank (as expected with no actual GPU uploads). This confirms that no higher-level code holds a direct dependency on the OpenGL API, and that introducing a new backend requires changes only within `Platform/`.

Chapter 6

Conclusion

This thesis set out to design and implement a simple 3D game engine in C++17 that demonstrates how the core subsystems of a real engine fit together, with particular attention to four engineering challenges: loading each asset exactly once, sequencing the manager subsystems in a dependency-respecting order, providing a flexible native scripting mechanism, and implementing a physically correct lighting model. All five goals stated in Chapter 1 have been met.

6.1 Summary of Contributions

The engine delivers a complete, runnable artefact. A user who clones the repository, satisfies the prerequisites (Visual Studio 2022, a Vulkan-capable GPU, the PhysX SDK), and builds the Sandbox project will see an interactive 3D scene with collision response, dynamic Blinn-Phong lighting, and C++ scripted behaviour running from the first frame.

The four focal contributions are:

Asset caching. The `AssetManager` guarantees that each file path is loaded from disk and uploaded to the GPU at most once per session. Reference counting through `shared_ptr` ensures that resources are freed automatically when no entity references them, without a manual release mechanism.

Manager sequence. Following the layered start-up and shut-down pattern advocated by Gregory [1], the five managers — `AssetManager`, `PhysicsManager`, `SceneManager`, `RenderManager`, and `UIManager` — are initialised in dependency order and shut down in reverse. The per-frame update sequence ensures that physics results are visible to scripts, and script modifications are visible to the renderer, within the same frame and without any one-frame lag.

Native scripting. The `ScriptableEntity` base class gives user scripts full access to the ECS through a single template method. The three Sandbox scripts — `FPSCameraScript`, `LanternFlickerScript`, and `RotatingCrateScript` — demonstrate input-driven movement, component animation, and

physics-state-aware behaviour respectively, all without any coupling between the scripting and rendering or physics subsystems.

Blinn-Phong lighting with UBOs. The fragment shader accumulates ambient, diffuse, and specular contributions from a directional light and up to four point lights. All per-frame lighting and camera data is uploaded to Uniform Buffer Objects once per frame, reducing the CPU-side OpenGL call count from $O(n_{meshes})$ to $O(1)$ regardless of scene size.

6.2 Reflections on Design Decisions

6.2.1 The Rendering Abstraction

The pure-virtual rendering abstraction achieved its stated goal: the entire Sandbox scene compiled and ran without any code above the `Platform/` directory holding a direct OpenGL dependency. However, the abstraction has not yet been validated by actually providing a second backend. A future implementation of `VulkanVertexBuffer` and its siblings would be a stronger proof of the claim than the stub test performed during evaluation.

The choice of virtual dispatch as the abstraction mechanism is simple and sufficient at the scale of this engine, but it does introduce one virtual call per draw-call boundary. A production engine at this scale might prefer a *type-erased* or *handle-based* resource system to avoid the vtable indirection on the hot path. For the purposes of this thesis, the virtual approach is the right trade-off between simplicity and extensibility.

6.2.2 EnTT and the ECS

EnTT proved to be an excellent library choice. Its sparse-set storage gave $O(1)$ component access without requiring any manual memory management, and its template-based API meant that all component lookups were resolved at compile time with no runtime overhead. The only limitation encountered was that EnTT's `view` iteration order is not guaranteed to be stable across insertions and removals; for deterministic replay or networked games this would require an additional ordering layer.

6.2.3 Physics Integration

The thin-wrapper approach to PhysX worked well for the scope of the engine. Hiding the entire PhysX API behind `Physics3DWorld` meant that none of the higher-level code needed to include any PhysX headers, and the integration points — `OnRuntimeStart` and `OnRuntimeStop` — were clean and testable. The main limitation is that the wrapper currently exposes only box colliders and rigid bodies. A more complete integration would expose capsule colliders for character controllers, joints for articulated objects, and the PhysX character controller for walking physics.

6.3 Future Work

Several directions for future development follow naturally from the current state of the engine.

A second rendering backend. The most direct validation of the interchangeable-API claim would be a Vulkan backend. The abstract resource classes are already defined; the work required is confined to implementing the concrete subclasses in a new `Platform/Vulkan/` directory.

A scene editor. The engine already includes an `EditorCamera` and Dear ImGui integration. A scene editor built on top of these would allow entities to be created, components to be edited, and scenes to be saved and loaded without recompiling, which is the next step toward a production-usable tool.

Expanded physics. Adding capsule colliders, mesh colliders, and the PhysX character controller would make the physics layer suitable for first-person games. The thin-wrapper design means these additions can be made without touching any engine code outside `Physics3D.cpp`.

A scripting language. The engine already vendors the Mono runtime. Replacing native C++ scripts with C# scripts compiled at runtime would allow game logic to be written and hot-reloaded without rebuilding the engine, which is the workflow used by Unity and Godot.

Render passes and post-processing. The current renderer writes directly to the default framebuffer. Introducing a framebuffer abstraction (already partially present in the codebase) would enable post-processing effects such as HDR tone-mapping, bloom, and screen-space ambient occlusion.

6.4 Closing Remarks

Building a game engine from first principles, even a small one, forces a depth of understanding that using a commercial engine cannot provide. Every design decision described in this thesis — where to place the abstraction boundary for the renderer, how to sequence the managers, how to expose the ECS to user scripts — had to be made deliberately and justified by the constraints of the problem. The result is a codebase that is small enough to read in a weekend but complete enough to run a non-trivial interactive scene, and that is, in the author's view, the most valuable property an educational engine can have.

Bibliography

1. GREGORY, Jason. *Game Engine Architecture*. 3rd. CRC Press, 2018.
2. EPIC GAMES. *Unreal Engine 5 Documentation* [`https://docs.unrealengine.com`]. 2024.
3. UNITY TECHNOLOGIES. *Unity Manual* [`https://docs.unity3d.com/Manual`]. 2024.
4. GODOT ENGINE CONTRIBUTORS. *Godot Engine: Free and open-source game engine* [`https://godotengine.org`]. 2024.
5. NYSTROM, Robert. *Game Programming Patterns*. Genever Benning, 2014.
6. MARTIN, Adam. Entity Systems are the Future of MMOG Development. 2007. Available also from: `http://t-machine.org`.
7. CAINI, Michele. *EnTT: A header-only, tiny and easy to use library for game programming* [`https://github.com/skypjack/entt`]. 2024.
8. GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
9. NVIDIA CORPORATION. *NVIDIA PhysX SDK Documentation* [`https://developer.nvidia.com/physx-sdk`]. 2023.
10. BENDERSKY, Jesse. *yaml-cpp: A YAML parser and emitter in C++* [`https://github.com/jbeder/yaml-cpp`]. 2024.
11. ASSIMP DEVELOPMENT TEAM. *Open Asset Import Library (Assimp)* [`https://github.com/assimp/assimp`]. 2024.
12. VRIES, Joey de. *LearnOpenGL*. Kendall & Welling, 2020.
13. GLFW CONTRIBUTORS. *GLFW: A multi-platform library for OpenGL development* [`https://www.glfw.org`]. 2024.
14. RICCIO, Christophe. *OpenGL Mathematics (GLM)* [`https://glm.g-truc.net`]. 2024.
15. CORNUT, Omar. *Dear ImGui: Bloat-free Graphical User Interface for C++* [`https://github.com/ocornut/imgui`]. 2024.
16. SELA, Gabi. *spdlog: Fast C++ logging library* [`https://github.com/gabime/spdlog`]. 2024.

Appendix A

Project Structure

Table A.1 lists the top-level directories of the engine source tree and the responsibility of each.

Table A.1: Top-level source tree layout of the GameEngine project.

Directory	**Contents**
`src/GameEngine/Core/`	Application, window, events, UUID, logging
`src/GameEngine/Scene/`	ECS wrappers, components, serialiser
`src/GameEngine/Renderer/`	Abstract GPU resources, Renderer3D, camera
`src/GameEngine/Physics/`	PhysX wrapper (`Physics3D.h/.cpp`)
`src/GameEngine/Managers/`	All five manager classes
`src/Platform/OpenGL/`	Concrete OpenGL implementations
`src/Platform/Windows/`	Windows window and input
`vendor/`	Third-party libraries (EnTT, PhysX, Assimp, ...)
`Sandbox/src/`	Demo application entry point
`Sandbox/src/Scripts/`	FPS camera, lantern flicker, rotating crate
`Sandbox/assets/`	Models, textures, shaders, scenes

Table A.2 lists the third-party libraries used by the engine, their purpose, and the version integrated.

Table A.2: Third-party libraries used by the engine.

Library	**Purpose**	**Reference**
EnTT	Entity-Component System storage	[7]
NVIDIA PhysX	Rigid-body physics simulation	[9]
Assimp	3D model import (OBJ, FBX, ...)	[11]
GLFW	OS window and input	[13]
Glad	OpenGL function loader	—
GLM	Mathematics (vectors, matrices)	[14]
Dear ImGui	Immediate-mode UI overlay	[15]
spdlog	Structured logging	[16]
yaml-cpp	Scene serialisation (YAML)	[10]
stb_image	PNG/JPG texture decoding	—